PULSEBOARD: APPLICATION PERFORMANCE MONITORING (APM) DASHBOARD

[GROUP NAME]

BITS ZC229T: Design Project

by

S.No	Student Name	BITS ID
1	[Student 1 - replace]	[BITS ID]
2	[Student 2 - replace]	[BITS ID]
3	[Student 3 - replace]	[BITS ID]
4	[Student 4 - replace]	[BITS ID]
5	[Student 5 - replace]	[BITS ID]

B.Sc. Design and Computing

Design Project work carried out at

[Organisation / Location]

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE PILANI (RAJASTHAN)

August 2026

PULSEBOARD: APPLICATION PERFORMANCE MONITORING (APM) DASHBOARD

[GROUP NAME]

BITS ZC229T: Design Project

by

S.No	Student Name	BITS ID
1	[Student 1 - replace]	[BITS ID]
2	[Student 2 - replace]	[BITS ID]
3	[Student 3 - replace]	[BITS ID]
4	[Student 4 - replace]	[BITS ID]
5	[Student 5 - replace]	[BITS ID]

B.Sc. Design and Computing

Design Project work carried out at

[Organisation / Location]

Submitted in partial fulfillment of the requirements for the
B.Sc. (Design and Computing) degree programme

Under the Supervision of

[Mentor Name and Designation]

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE
PILANI (RAJASTHAN)

August 2026

CERTIFICATE

This is to certify that the Design Project entitled PULSEBOARD: APPLICATION PERFORMANCE MONITORING (APM) DASHBOARD and submitted by [STUDENT NAME] having BITS ID No. [BITS ID] for the partial fulfillment of the requirements of the B.Sc. (Design and Computing) degree programme of BITS embodies the bona fide work done by the student under my supervision.

_____

Signature of the Mentor

[Mentor Name]

[Designation / Department]

Place: _____

Date: _____

Note: copy this certificate page once for each group member, if your institution requires individual certificates.

Birla Institute of Technology & Science, Pilani
Work-Integrated Learning Programmes Division

BITS ZC229T: Design Project

ABSTRACT

BITS ID No.	[BITS ID]
NAME OF THE STUDENT	[Student Name]
EMAIL ADDRESS	[Student Email]
STUDENT'S EMPLOYING ORGANIZATION & LOCATION	[Organisation and Location]
MENTOR'S NAME	[Mentor Name]
MENTOR'S EMAIL ADDRESS	[Mentor Email]
DESIGN PROJECT TITLE	PULSEBOARD: APPLICATION PERFORMANCE MONITORING (APM) DASHBOARD

ABSTRACT: PulseBoard is a web-based Application Performance Monitoring dashboard developed to convert application telemetry into an accessible operational view. The system reads a structured APM dataset, derives response-time, resource, availability, error-rate and throughput metrics, and presents them through interactive filters, KPI cards, trend charts, status views and an alert table. A Node.js and Express backend exposes overview and alert endpoints; a React and Vite frontend renders the monitoring experience with reusable components and Recharts visualizations. The project demonstrates a practical path from CSV records to service-level insight. Across 14,400 observations for ten applications, the dashboard identifies a warning-level average response time (359.99 ms) and a warning-level average error rate (1.31%) while availability remains healthy at 99.74%.

Broad Academic Area of Work: Application Performance Monitoring, Web Development and Data Visualization

Key words: APM, observability, React, Express, REST API, Recharts, CSV analytics, alerting, service health, dashboard.

_____ **Signature of the Student** **Name: [Student Name]** **Date: [Date]** **Place: [Place]**	_____ **Signature of the Mentor** **Name: [Mentor Name]** **Date: [Date]** **Place: [Place]**

ACKNOWLEDGEMENTS

We express our sincere gratitude to our mentor, faculty members and the BITS Pilani WILP team for their guidance and encouragement during the development of PulseBoard. We also acknowledge the support of our organisation and peers, whose feedback helped refine the dashboard's usability, data presentation and operational focus.

TABLE OF CONTENTS

Page numbers are indicative and will update if sections are added or removed in Word.

LIST OF FIGURES AND TABLES

Figures

Tables

Source note: figures are generated from the repository's APM dataset and the implemented component structure.

1. INTRODUCTION

1.1 Background

Modern organisations depend on many distributed applications, each of which produces operational signals such as response time, CPU utilisation, memory consumption, availability, request volume and error counts. When these signals are dispersed across files or tools, it becomes difficult for stakeholders to recognise degradation early and compare the health of services consistently. Application Performance Monitoring (APM) addresses this visibility problem by collecting, deriving and presenting health-oriented measures in a form that supports timely action.

1.2 Problem Statement

The project addresses the challenge of turning a large set of application telemetry records into a single, understandable monitoring view. A reviewer should be able to select an application, region and reporting window, understand its current operating posture, compare services and identify potential incidents without manually calculating averages or searching raw CSV files.

1.3 Objectives

- Build an interactive dashboard for summarising application performance and service health.
- Calculate metrics for latency, CPU, memory, availability, error rate, throughput and HTTP response counts.
- Classify metric values as Healthy, Warning or Critical using transparent thresholds.
- Provide filters and charts that support comparison across applications and regions.
- Expose reusable overview and alert endpoints for a React-based client.

1.4 Scope

In scope are CSV-based monitoring data, backend metric aggregation, health classification, basic alert generation, a responsive dashboard user interface and automated endpoint checks. The current implementation is a data-driven monitoring prototype: it does not ingest live telemetry streams, store historical data in a database, authenticate users or perform automated remediation.

2. LITERATURE REVIEW / RELATED WORK

2.1 Monitoring Dashboards

Monitoring dashboards are commonly organised around service indicators that answer practical questions: Is the system available? Are requests responding within acceptable latency? Is infrastructure under pressure? Are errors increasing? A dashboard is most useful when it combines overview metrics with drill-down information and retains the context required for a reader to decide what to investigate next. PulseBoard follows this principle by pairing KPI cards with application comparisons, time-series views, detailed summary rows and alerts.

2.2 Web Technology Context

The frontend uses React's component model for reusable UI elements and Vite for development and production build handling. The backend uses Express to define HTTP routes and middleware around a small REST API. Recharts provides chart components for the React screen, while Papa Parse converts the source CSV into JavaScript objects. This combination is appropriate for a prototype because it separates data preparation, API logic and presentation without requiring a complex deployment platform.

2.3 Contribution of the Project

- A coherent APM workflow from CSV source data to browser-based decision support.
- A reusable calculation layer for total metrics, per-application metrics and time-bucket summaries.
- Transparent health thresholds that keep alert meaning consistent across dashboard widgets.
- A frontend that retains a fallback sample view if the live API is temporarily unavailable.

3. REQUIREMENT ANALYSIS

3.1 Stakeholders and User Needs

The primary users are operations analysts, application owners and project reviewers. They need rapid visibility into overall health, the ability to compare services, evidence of performance trends and an unambiguous way to see which measures require attention. Developers additionally require a small, testable backend contract that can be consumed by the user interface.

3.2 Functional Requirements

ID	Requirement
FR-01	Load the provided CSV dataset and parse it into structured records.
FR-02	Filter rows by application, region, environment, severity, event, health status and reporting window.
FR-03	Calculate dataset-wide and per-application performance indicators.
FR-04	Return overview data, chart data and alert data through REST endpoints.
FR-05	Render KPI cards, filters, charts, tables and alert information in the frontend.
FR-06	Show fallback dashboard data when the backend cannot be reached.

Table 1. Functional requirements

3.3 Non-Functional Requirements

Category	Requirement
Usability	The dashboard should use clear labels, status colours and readable summaries.
Performance	Data processing should complete quickly for the supplied 14,400-record dataset.
Reliability	Endpoints and the build process should be verified by automated tests and a production build.
Maintainability	Backend calculations and frontend widgets should be separated into focused modules.

Table 2. Non-functional requirements

4. SYSTEM DESIGN

4.1 Architecture

PulseBoard uses a lightweight three-part architecture. A CSV file is the data source; the Express backend reads and filters records, calculates aggregate values and returns JSON; the React frontend requests this data and renders it through reusable components. This design keeps metric logic on the server while allowing the client to focus on interaction and data presentation.

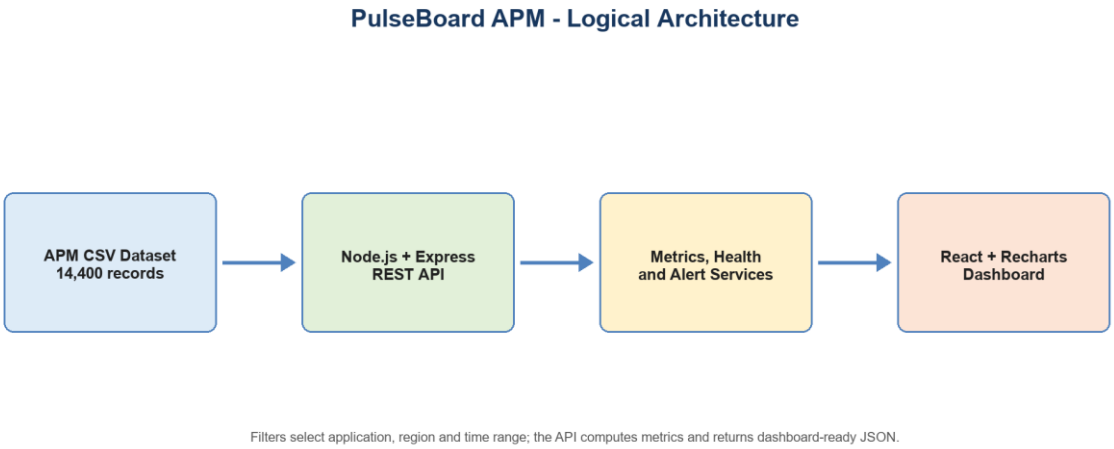

Figure 1. PulseBoard logical architecture

4.2 Core Modules

Module	Responsibility
CSV Service	Reads the local data file, parses CSV rows and applies filters.
Metrics Service	Calculates averages, totals, p95 latency, per-application metrics and time buckets.
Health Service	Maps calculated metric values to health labels using threshold rules.
Alert Service	Produces concise alerts where a calculated health state needs attention.
React Components	Render filters, KPI cards, charts, tables, status views and navigation.

Table 3. Core backend and frontend modules

4.3 Data Design

Each dataset record captures a timestamp, application, environment, region, performance values, HTTP counts, resource measures, event metadata and a health status. The current dataset covers 14,400 records from 1 May 2026 to 30 May 2026. It includes ten applications, three regions and both production and UAT environments. Numeric fields are converted to numbers before aggregation, while categorical fields form the basis of filter choices and frequency analysis.

Entity / Group	Key Content
Application telemetry	Timestamp, application, environment, region and event context.
Performance measures	Response time, p95 latency, throughput, availability and error rate.
Resource measures	CPU usage, memory usage and active users.
HTTP diagnostics	4xx and 5xx response counts used to give error context.
Health output	Healthy, Warning or Critical labels derived from defined threshold rules.

Data groups used by the monitoring model

4.4 Dashboard Design

The dashboard is organised as a short path from context to action: filters establish the slice of data; a hero section explains the active focus; KPI cards show primary measures; charts compare performance; and tables list summary results and alerts. This arrangement lets a reader scan the most important state before moving to detailed interpretation.

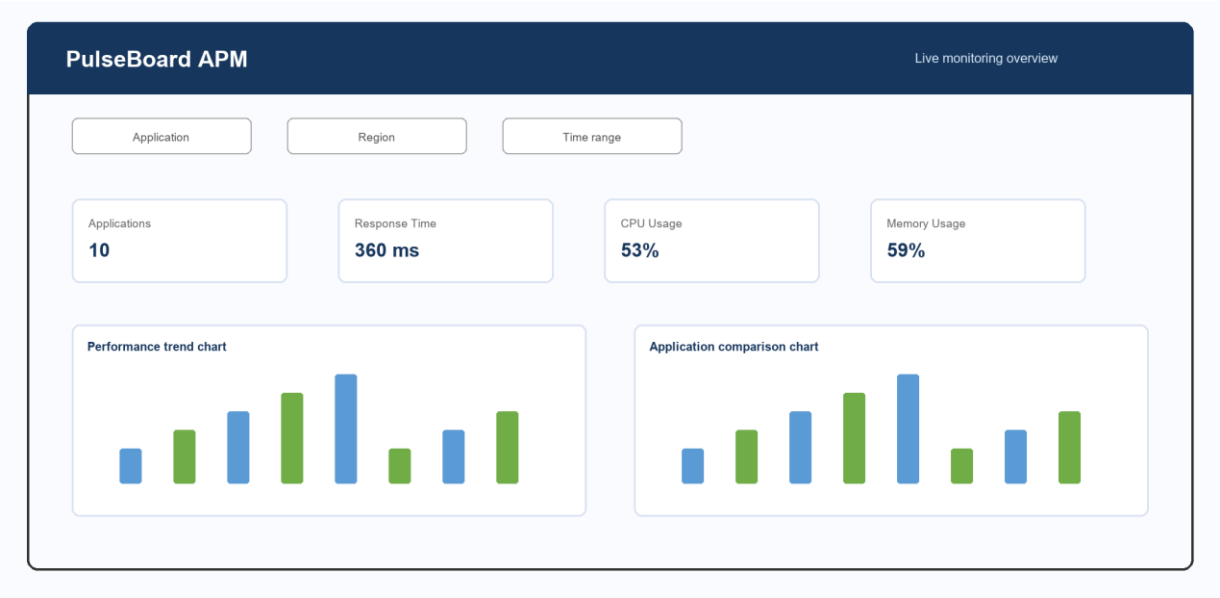

Figure 4. PulseBoard dashboard component layout (derived from the implemented UI)

5. IMPLEMENTATION

5.1 Technologies Used

Technology	Role in PulseBoard
React	Component-based frontend rendering and state handling.
Vite	Frontend development server and production build pipeline.
Node.js + Express	Backend runtime and HTTP REST endpoints.
Papa Parse	CSV parsing with header and dynamic typing options.
Recharts	Composable charting elements for the React dashboard.
Jest + Supertest	Automated API testing for the backend.

Technology stack used in the implementation

5.2 Backend API

The Express server exposes a root health-style route, an overview route and an alerts route. The overview endpoint loads the CSV, builds the unique filter values, applies query-based selections and returns metrics, health indicators, chart data and alerts in one payload. The alerts endpoint returns the active alert list with timestamps. The API is deliberately compact so the frontend can obtain the data required for a monitoring view without joining several endpoints.

Endpoint	Purpose
GET /	Returns the project name and backend running status.
GET /api/overview	Returns filters, overall metrics, health, alerts, application metrics and time-series data.
GET /api/alerts	Returns the derived active alerts with time metadata.

PulseBoard REST endpoints

5.3 Metric and Health Logic

Metric values are calculated from the currently filtered record set. The implementation calculates averages for response time, CPU, memory, availability, error rate and throughput; sums HTTP 4xx and 5xx counts; and derives p95 latency from the sorted latency values. The health service applies fixed thresholds. The same labels are reused by the KPI cards, overview data and alert logic, which prevents different widgets from interpreting the same value differently.

Measure	Health Classification
CPU usage	Healthy < 70%; Warning 70-90%; Critical > 90%
Memory usage	Healthy < 75%; Warning 75-90%; Critical > 90%
Response time	Healthy < 300 ms; Warning 300-500 ms; Critical > 500 ms
Availability	Healthy >= 99%; Warning 98-99%; Critical < 98%
Error rate	Healthy <= 1%; Warning > 1-5%; Critical > 5%

Table 4. Threshold-based health classification

5.4 Frontend Behaviour

The React application maintains selections for application, region and time range. When the selection changes, it requests a refreshed overview payload. Data is transformed into card values, summary rows, chart data and ordered alerts. The dashboard also includes a fallback data object. This makes the screen demonstrable if the deployed backend is unavailable, while an explicit status message differentiates sample data from a live API response.

5.5 Dataset Profile

The supplied dataset contains 14,400 records representing 10 applications across 3 regions. The majority of records are from Production (12,981), with a smaller UAT subset (1,419). The data is sufficient to show aggregate and per-application comparisons without introducing a database dependency.

6. TESTING AND EVALUATION

6.1 Test Strategy

Testing focused on the server contract and the frontend production build. Backend unit/integration tests use Jest and Supertest to request the routes from the Express application directly. The build step runs Vite's production bundle process, confirming that the frontend components and imports resolve correctly. This provides repeatable evidence for core functionality, although it does not replace manual usability testing on a deployed system.

ID	Test Area	Expected Result	Status
T-01	Root endpoint	Project status JSON is returned with HTTP 200.	**Passed**
T-02	Overview endpoint	Metrics, health and alert fields are present with HTTP 200.	**Passed**
T-03	Alerts endpoint	An alerts array is returned with HTTP 200.	**Passed**
T-04	Frontend build	Vite production build completes without module or compilation errors.	**Passed**

Table 5. Test execution summary

6.2 Test Result

The CI-style local verification completed successfully: three backend tests passed and the React application built successfully for production. The generated build included the HTML entry point, a CSS asset and a JavaScript bundle, demonstrating that the current codebase compiles and packages as expected.

6.3 Evaluation Limits

The reported result verifies the application in a local development context using a static dataset. Before production use, the system should be evaluated against real-time traffic, large data volumes, network latency, authentication requirements and alert-delivery integrations. A database-backed history and observability instrumentation would also be needed for longitudinal analysis.

7. RESULTS AND DISCUSSION

7.1 Dataset-Wide Operational Indicators

Indicator	Observed Value	Interpretation
Average response time	**359.99 ms**	Warning (>= 300 ms)
P95 latency	**1245 ms**	Tail latency needs investigation
Average CPU usage	**52.89%**	Healthy
Average memory usage	**58.94%**	Healthy
Average availability	**99.74%**	Healthy (>= 99%)
Average error rate	**1.31%**	Warning (> 1%)
Average throughput	**9095.33 rpm**	Contextual demand measure
HTTP 4xx / 5xx counts	**142,797 / 194,651**	Diagnostic volume

Table 6. Dataset-wide operational indicators

The aggregate values classify CPU, memory and availability as healthy. However, the response-time average and error-rate average fall into the dashboard's warning band. Therefore, the dashboard's most useful operational function in this dataset is to direct attention toward latency and error contributors while showing that resource utilisation is not the immediate broad constraint.

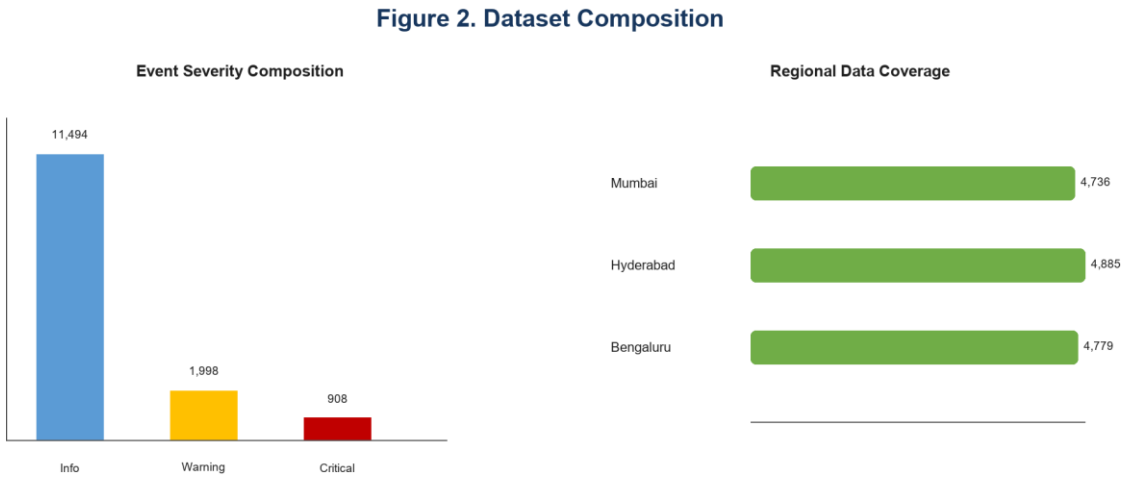

Figure 2. Dataset composition

7.2 Application-Level Comparison

All ten applications report warning-level average response times under the configured threshold policy. Flipkart has the highest average response time at 412.18 ms, followed by Payment Gateway at 407.22 ms. The Payment Gateway's role in transaction completion makes it a high-priority candidate for closer review, despite the dataset-wide availability remaining healthy. Citrix Workspace has the highest p95 latency at 1,329 ms, suggesting pronounced slow-tail behaviour for at least some requests.

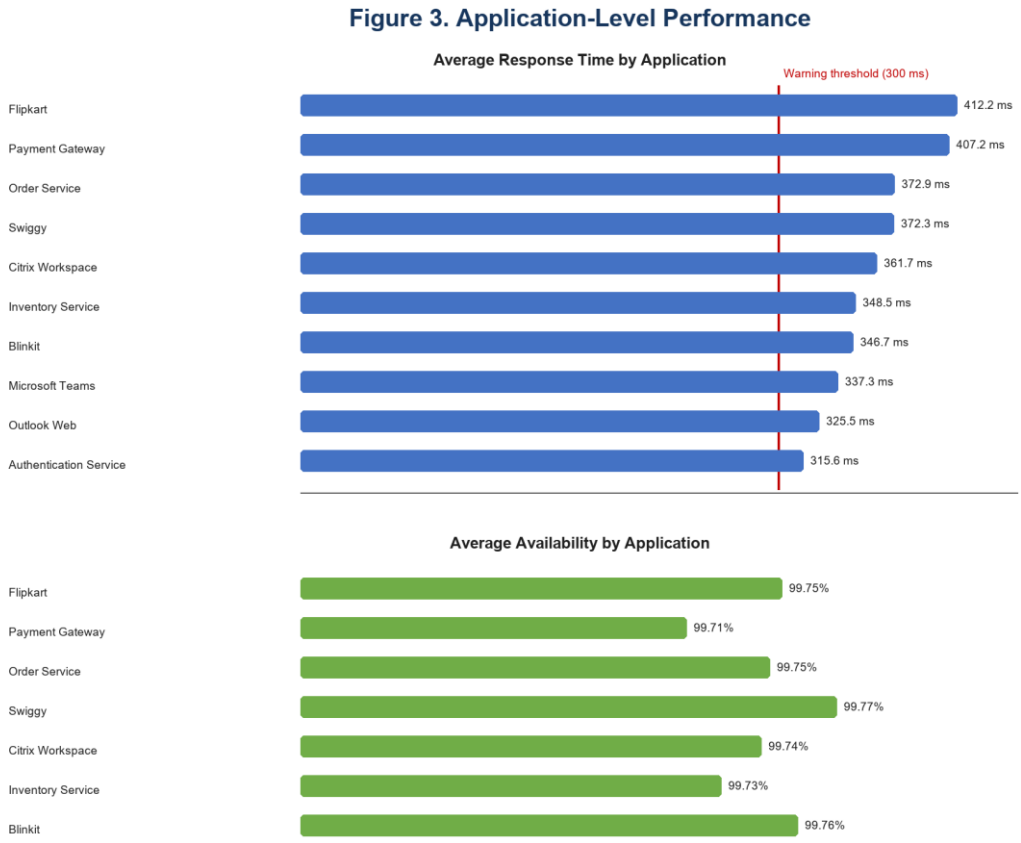

Figure 3. Application-level performance

7.3 Discussion

The results illustrate why a single average should not be treated as the entire story. The average response time establishes a broad warning state, but the application comparison identifies the services that are slower relative to their peers. Availability values form a narrow healthy range, while latency and error measures carry more discriminating information. The dashboard allows these signals to be viewed together so that an analyst can form a triage hypothesis before examining logs, traces or external dependencies.

8. INFERENCES / SUMMARY

- The supplied data is broad enough to compare ten services while preserving region, environment and event context.
- Healthy average CPU, memory and availability do not eliminate a user-impacting latency or error issue.
- Response-time and error-rate thresholds mark the overall dataset as Warning, making these the initial investigation priorities.
- A component-based frontend and a modular calculation layer make the prototype easier to adapt for additional filters and live sources.
- The current dashboard is suitable for explanatory monitoring and data exploration; production operations would require persistent storage, access control and live ingestion.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

PulseBoard demonstrates an end-to-end APM dashboard workflow using a real project dataset and a contemporary web stack. It reads monitoring records, computes performance and health information, exposes a compact API and renders an interactive view for filtering, comparison and alert awareness. Local tests and a production frontend build both complete successfully. The project meets its aim of making telemetry-derived operational insight more accessible than raw CSV inspection.

9.2 Future Work

- Ingest real-time metrics through agents, message queues or observability APIs.
- Store telemetry in a time-series database and apply timestamp-aware range filtering.
- Add authentication, role-based access and deployment-safe environment configuration.
- Integrate logs and distributed traces to support root-cause analysis from an alert.
- Add alert acknowledgement, notification channels and incident history.
- Extend the test suite with filter-specific, error-path and end-to-end browser tests.

10. REFLECTION AND LEARNING

10.1 Key Learnings

- A clear metric contract makes a dashboard more reliable because calculations and health labels are shared across views.
- Data visualisation should support a monitoring question, not merely display available fields.
- Separating CSV parsing, metric aggregation, health classification and alert creation improves maintainability.
- Fallback UI states are valuable for demos, but must clearly disclose when data is not live.
- Automated endpoint testing and a production build test provide a useful baseline before deployment.

10.2 Challenges

The project required aligning diverse performance measures onto a simple three-level health model without masking important differences. Another practical challenge was converting a static CSV data source into an interface that behaves like a monitoring product while accurately signalling the boundary between sample/fallback information and a reachable backend API.

10.3 Improvements

The next iteration should prioritise true timestamp parsing, a persistent data store and live telemetry ingestion. These changes would make time-range filtering semantically precise and support longitudinal health reporting. They would also enable better alert history, root-cause navigation and production-level reliability testing.

11. BIBLIOGRAPHY

11. PulseBoard Project Source Code and APM Dataset. Provided project repository, accessed August 2026.
12. React Documentation. React Team. https://react.dev/
13. Vite Documentation - Getting Started. VoidZero. https://vite.dev/guide/
14. Express.js Documentation. OpenJS Foundation and Express contributors. https://expressjs.com/
15. Papa Parse Documentation. https://www.papaparse.com/docs
16. Recharts Documentation. https://recharts.github.io/
17. Jest Documentation. https://jestjs.io/
18. SuperTest Repository Documentation. https://github.com/forwardemail/supertest

12. REFERENCES

All implementation claims in this report are based on the supplied PulseBoard repository. Technology descriptions are supported by the official documentation listed above. Dataset-wide figures and conclusions were calculated from backend/data/PulseBoard_APM_14400_Dataset_With_HTTP_Status.csv using the same aggregation approach as the project backend.

13. APPENDICES

Appendix A. Project Structure

Path	Purpose
backend/server.js	Express application and API endpoints.
backend/services/csvService.js	Dataset parsing and filter functions.
backend/services/metricsService.js	Metric, p95 and application-level calculations.
backend/services/healthService.js	Health classification wrapper.
backend/services/alertService.js	Threshold-based alert generation.
frontend/src/App.jsx	Application state, data fetching and dashboard assembly.
frontend/src/components	Reusable filters, cards, charts, tables and navigation.
backend/__tests__/overview.test.js	API checks executed through Jest and Supertest.

Appendix table A1. Important source files

Appendix B. Final Submission Checklist

- Replace all bracketed student, ID, mentor, organisation, place and date fields.
- Add or duplicate individual certificate and abstract pages for every group member as required.
- Confirm the institution name, course code and classification statement against your official template.
- Update the Table of Contents page numbers after any final edits in Word.
- Obtain mentor and student signatures before formal submission.